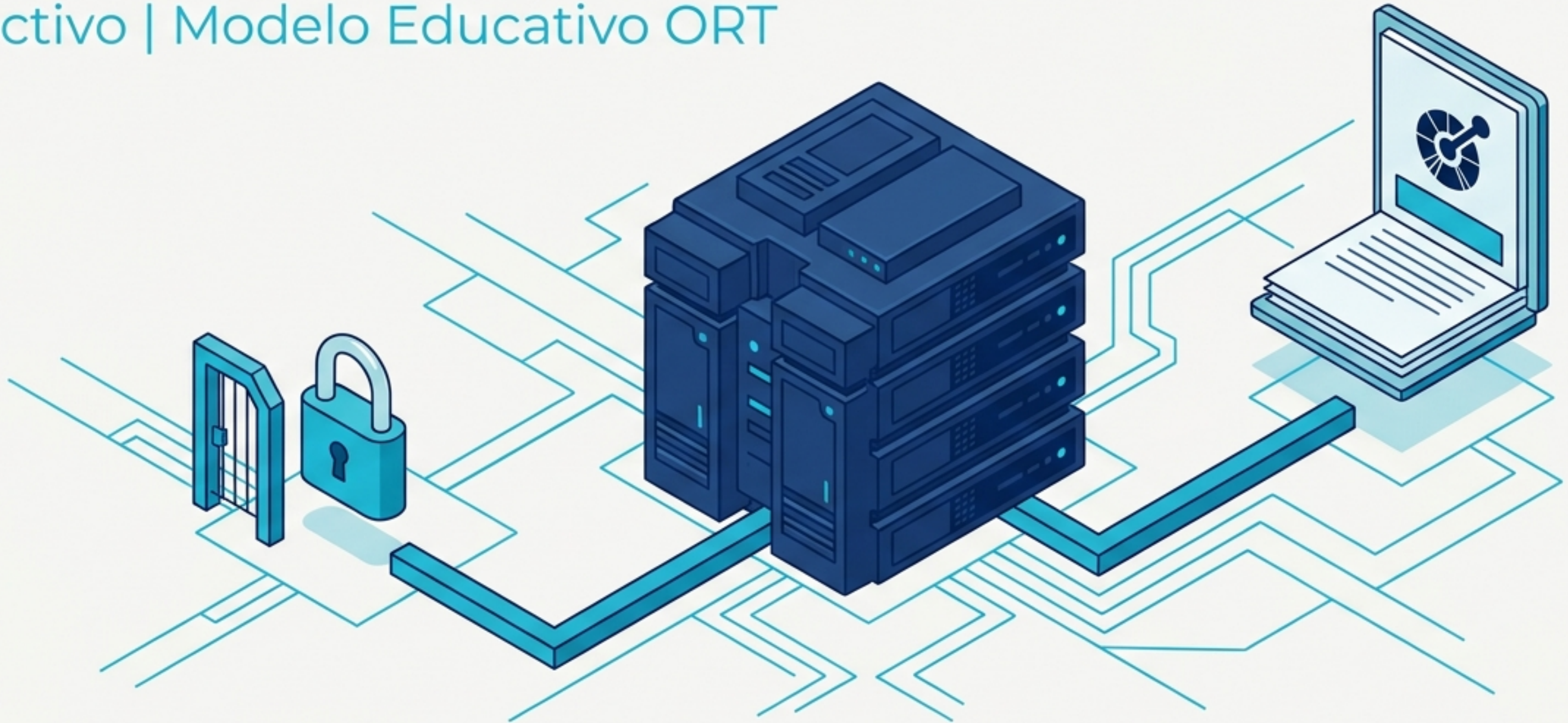
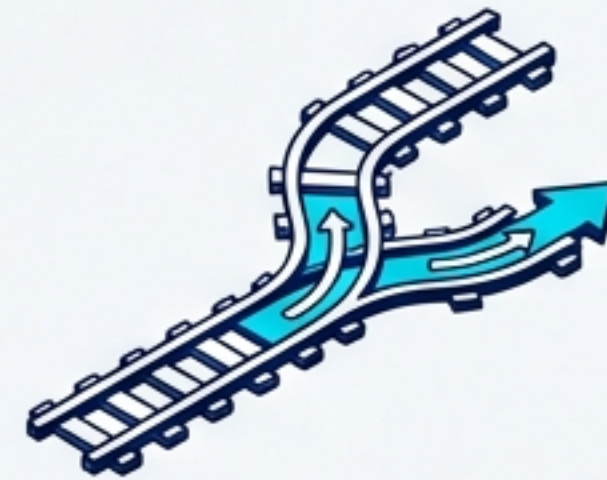
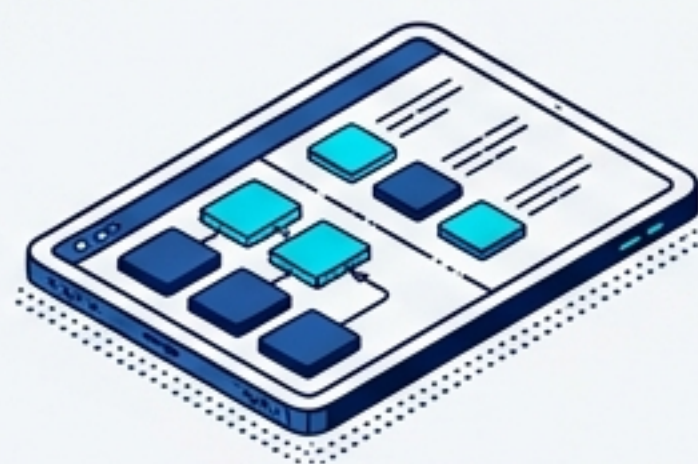


Desarrollo Backend: Persistencia, Seguridad y Documentación

Ciclo Lectivo | Modelo Educativo ORT



El camino de hoy: de la memoria volátil a una API profesional



1. Historial JSON

Guardar conversaciones sin bases de datos complejas.

2. Seguridad y Accesos

Autenticación y Autorización.

3. Documentación

OpenAPI, Swagger y Scalar.

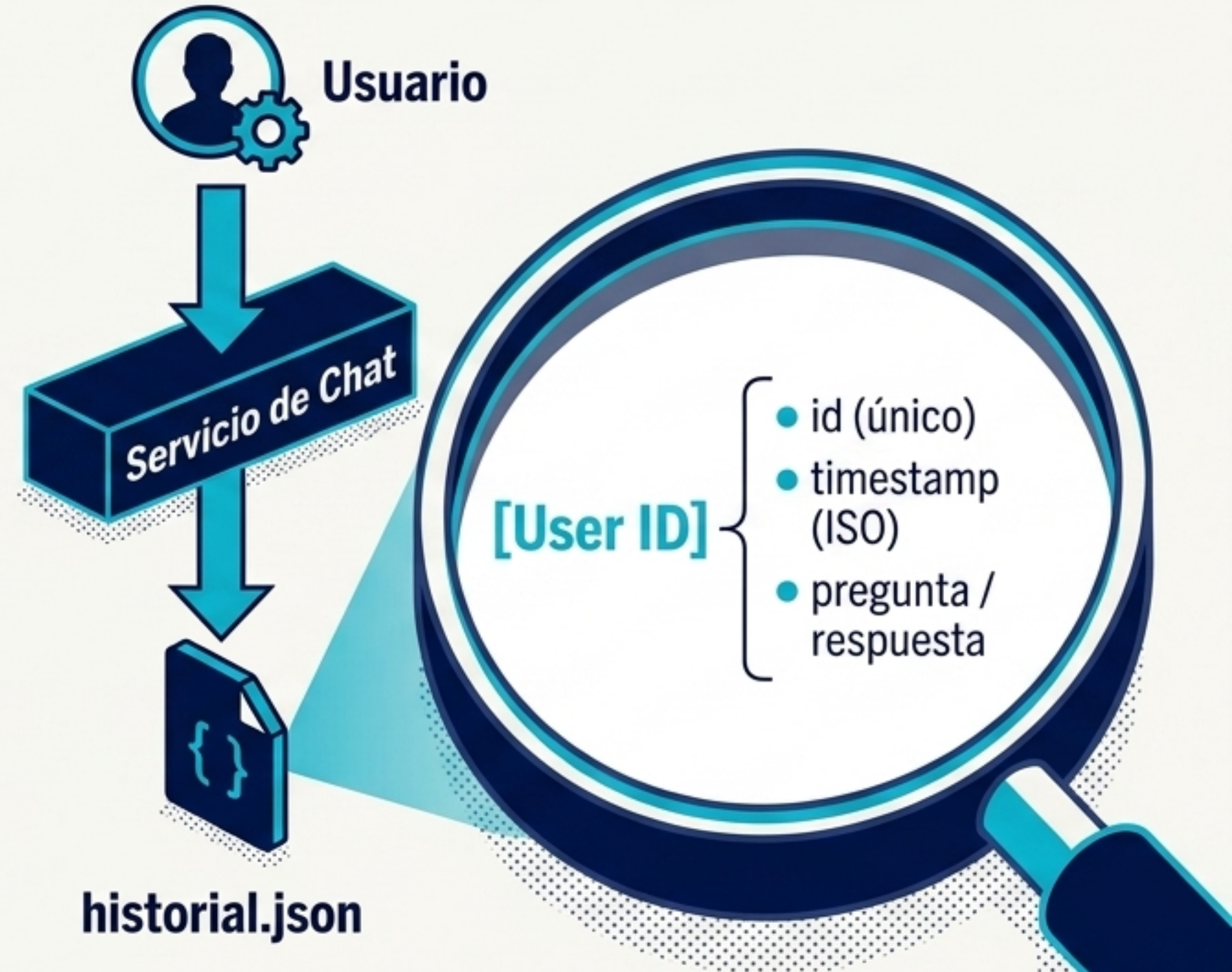
4. Versionado

Evolucionar sin romper clientes existentes.

Simulación de Base de Datos: El poder de un simple archivo JSON

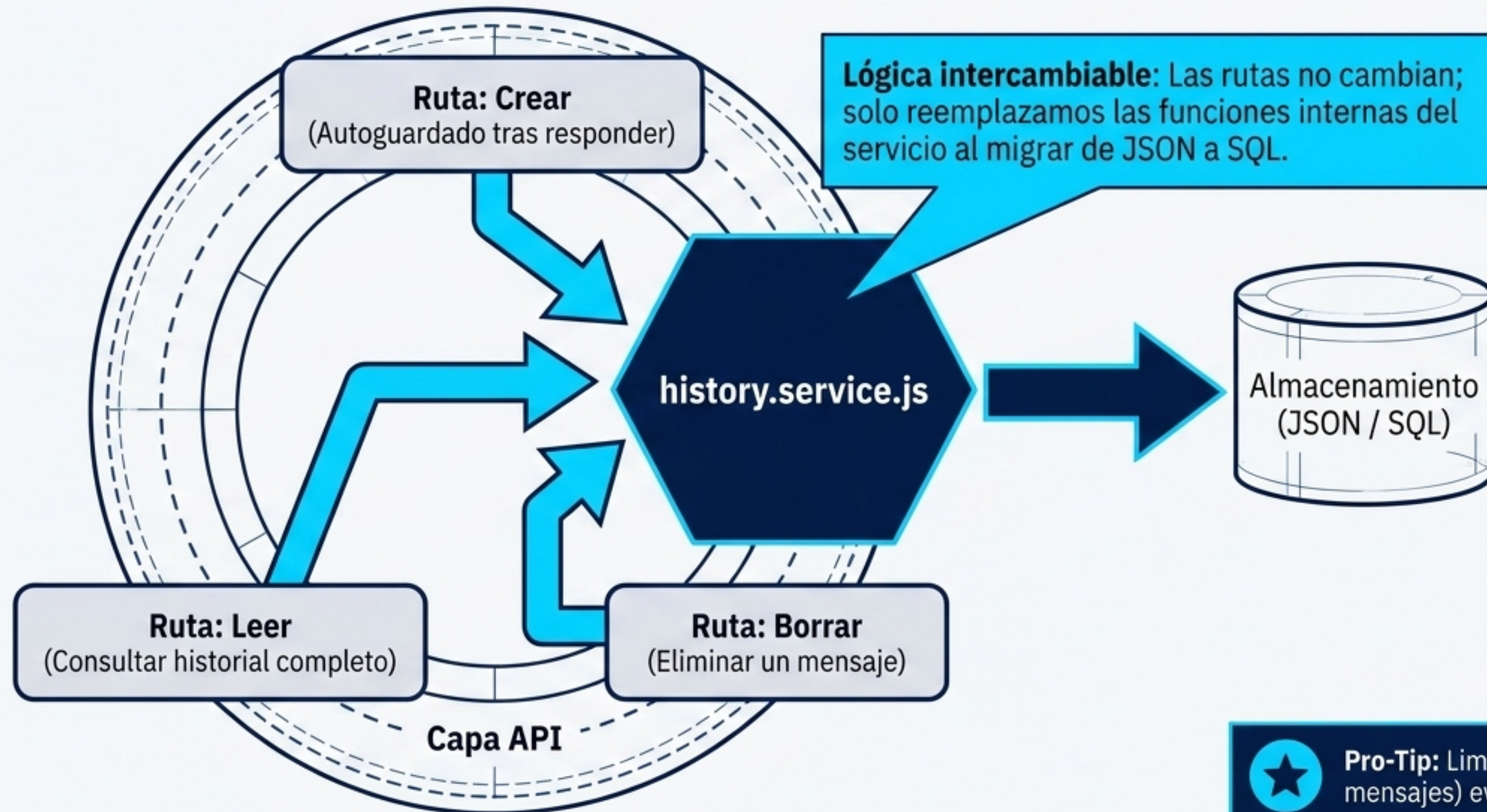
Iterar rápido manteniendo la lógica de negocio intacta

- Sin dependencias externas ni configuraciones de red.
- Simula operaciones CRUD reales de manera local.
- Mantiene la simplicidad operativa en etapas tempranas.



Abstracción: El Servicio de Historial como puente

Preparando el terreno para una futura migración a SQL o Supabase



El control de acceso: Identidad vs. Permisos

Una API abierta es un peligro; nadie entra sin identificarse

Autenticación
¿Quién eres?



Autorización
¿Qué puedes hacer?



Regla de negocio: Solo el dueño legítimo de un historial (Identidad) tiene el poder de borrarlo (Permisos).

Matriz de decisión de seguridad

JWT frente a API Keys: Herramientas diferentes para actores diferentes

Característica	X-API-KEY	JSON Web Token (JWT)
Uso Ideal	Scripts, Máquina a Máquina (M2M)	Usuarios reales (Logueados)
Estado en Servidor	No	No (Stateless)
Identifica Usuario	No	Sí (A través del Payload)
Expiración	Manual	Automática (Corta vida)
Revocación	Regenerar la clave entera	Lista negra o expiración

ESTRATEGIA DEL PROYECTO: Usamos JWT para proteger el Chat (usuarios) y API Keys para los endpoints de Administración interna.

El escudo protector: Rate Limiting

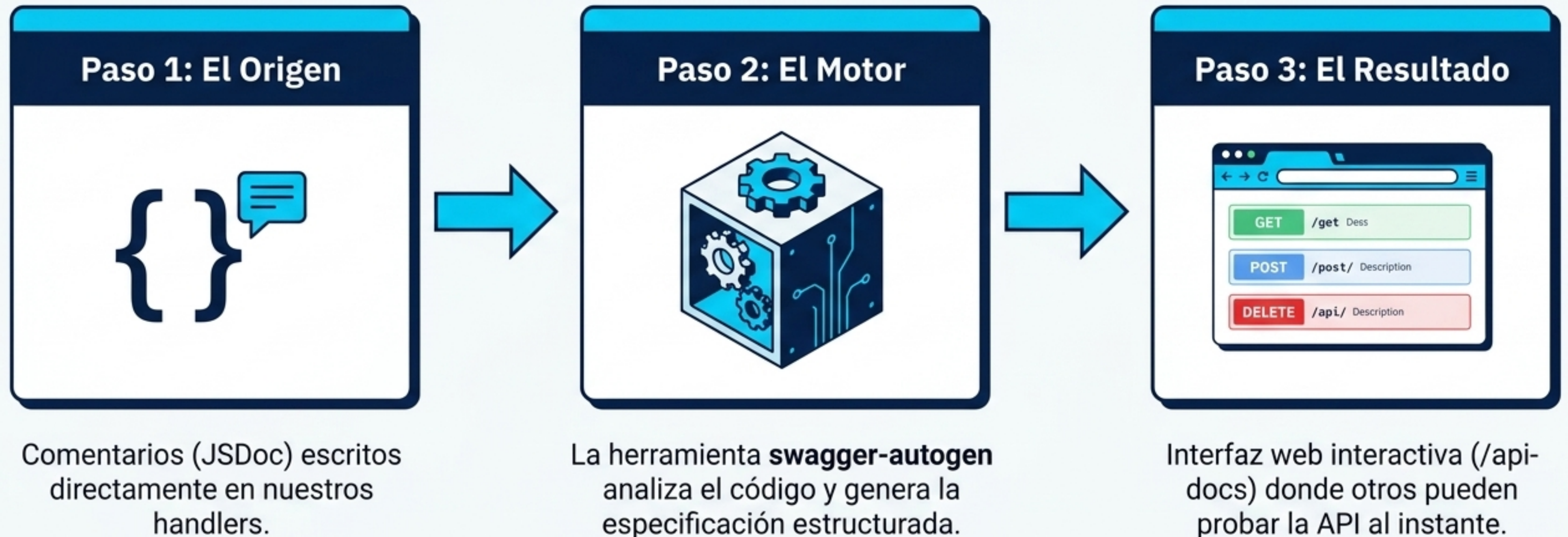
Controlando el tráfico para garantizar disponibilidad



El Rate Limiting previene la caída del sistema limitando la cantidad de requests que un usuario puede hacer por segundo.

El contrato de la API: Manuales Interactivos

Implementando OpenAPI con Swagger y Scalar



Evolución segura: Versionado a través de rutas

Cómo mejorar el sistema sin romper los clientes existentes



Regla de oro: Solo creamos una nueva versión cuando introducimos cambios que rompen la compatibilidad (campos renombrados, tipos de datos modificados, o endpoints eliminados).

Manos a la obra: Desafíos para la próxima clase

Integrando persistencia, seguridad y documentación en el proyecto



1. Flujo de Autenticación: Implementar el Login (POST /auth/login -> Obtener Token -> Usarlo en POST /chat).



2. Generar el Contrato: Documentar al menos 3 endpoints utilizando swagger-autogen.



3. Investigación de Arquitectura: ¿Qué es un Refresh Token, dónde se guarda (HttpOnly), y por qué es vital para la seguridad?



Bonus Challenge: Implementar un middleware que registre cada request (método, ruta, status, tiempo).

¡Éxitos en el desarrollo!